

WEEK 3 MODULE

Part 1: Handwritten Notes

Week 3 Module

Topic 1: Merge Sort

- Concept: Merge sort works by repeatedly dividing the array into smaller subarrays until each subarray contains a single element.
- Merging: These smaller subarrays are then merged in a sorted manner to produce the final sorted array.
- Use cases: It is highly effective when an efficient sorting technique is required for very large datasets, such as organizing large numbers of product prices or employee salary records.
- Time complexity: $O(N \log N)$ in all cases (worst, average and best), making it highly reliable for massive inputs.

Topic 2: Quick Sort

- Concept: Quick sort is an efficient algorithm often used to process large datasets quickly, such as race timings or large lists of library book IDs.
- Methodology: It uses a "divide and conquer" approach by selecting a 'pivot' element and positioning the other elements into two sub-arrays according to whether they are less than or greater than the pivot.
- Time complexity: $O(N \log N)$ on average, though it can degrade to $O(N^2)$ in the worst case if the pivot is chosen poorly on already sorted data.

Part 2: Merge Sort Programs

1. Rank Students by Marks

```
#include <stdio.h>
void merge(int arr[], int l, int m, int r)
{
    int n1 = m - l + 1;
    int n2 = r - m;
    int L[n1], R[n2];

    for (int i = 0; i < n1; i++) L[i] = arr[l + i];
    for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2)
    {
        if (L[i] <= R[j])
        {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
    while (i < n1) { arr[k] = L[i]; i++; k++; }
    while (j < n2) { arr[k] = R[j]; j++; k++; }
}
void mergeSort(int arr[], int l, int r)
{
    if (l < r)
    {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}
int main()
{
    int n;
    scanf("%d", &n);
    int marks[n];
    for (int i = 0; i < n; i++) scanf("%d", &marks[i]);
    mergeSort(marks, 0, n - 1);
    for (int i = 0; i < n; i++) printf("%d ", marks[i]);
    printf("\n");
    return 0;
}
```

Output

6

76 98 26 88 36 49

26 36 49 76 88 98

=== Code Execution Successful ===

2. Organizing Product Prices

```
#include <stdio.h>
void merge(int arr[], int l, int m, int r)
{
    int n1 = m - l + 1;
    int n2 = r - m;
    int L[n1], R[n2];

    for (int i = 0; i < n1; i++) L[i] = arr[l + i];
    for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2)
    {
        if (L[i] <= R[j])
        {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
    while (i < n1) { arr[k] = L[i]; i++; k++; }
    while (j < n2) { arr[k] = R[j]; j++; k++; }
}

void mergeSort(int arr[], int l, int r) {
    if (l < r)
    {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

int main()
{
    int n;
    scanf("%d", &n);
    int prices[n];
    for (int i = 0; i < n; i++) scanf("%d", &prices[i]);

    mergeSort(prices, 0, n - 1);

    for (int i = 0; i < n; i++) printf("%d ", prices[i]);
    printf("\n");
    return 0;
}
```

Output

5

20 05 10 40 35

5 10 20 35 40

=== Code Execution Successful ===

3. Sort Employee Records by Salary

```
#include <stdio.h>
void merge(int arr[], int l, int m, int r)
{
    int n1 = m - l + 1;
    int n2 = r - m;
    int L[n1], R[n2];

    for (int i = 0; i < n1; i++) L[i] = arr[l + i];
    for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2)
    {
        if (L[i] <= R[j])
        {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
    while (i < n1) { arr[k] = L[i]; i++; k++; }
    while (j < n2) { arr[k] = R[j]; j++; k++; }
}

void mergeSort(int arr[], int l, int r)
{
    if (l < r)
    {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

int main()
{
    }
```

```
int n;
scanf("%d", &n);
int salaries[n];
for (int i = 0; i < n; i++) scanf("%d", &salaries[i]);

mergeSort(salaries, 0, n - 1);

for (int i = 0; i < n; i++) printf("%d ", salaries[i]);
printf("\n");

printf("Lowest Salary: %d\n", salaries[0]);
printf("Highest Salary: %d\n", salaries[n - 1]);
printf("Difference: %d\n", salaries[n - 1] - salaries[0]);
return 0;
}
```

Output

5

1009 2057 1002 4629 7283

1002 1009 2057 4629 7283

Lowest Salary: 1002

Highest Salary: 7283

Difference: 6281

=== Code Execution Successful ===

Part 3: Quick Sort Programs

1. Race Timing Analysis

```
#include <stdio.h>
void swap(int* a, int* b)
{
    int t = *a;
    *a = *b;
    *b = t;
}

int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = (low - 1);
    for (int j = low; j <= high - 1; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```



```
int main()
{
    int n;
    scanf("%d", &n);
    int timings[n];
    for (int i = 0; i < n; i++) scanf("%d", &timings[i]);

    quickSort(timings, 0, n - 1);

    for (int i = 0; i < n; i++) printf("%d ", timings[i]);
    printf("\n");
    return 0;
}
```

Output

```
7
02 11 06 01 16 22 19
1 2 6 11 16 19 22
```

```
=== Code Execution Successful ===
```

2. Organizing Library Book IDs

```
#include <stdio.h>
void swap(int* a, int* b)
{
    int t = *a;
    *a = *b;
    *b = t;
}

int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = (low - 1);
    for (int j = low; j <= high - 1; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main()
```

```
{  
    int n;  
    scanf("%d", &n);  
    int ids[n];  
    for (int i = 0; i < n; i++) scanf("%d", &ids[i]);  
  
    quickSort(ids, 0, n - 1);  
  
    for (int i = 0; i < n; i++) printf("%d ", ids[i]);  
    printf("\n");  
    return 0;  
}
```

Output

```
4  
8363 0001 1000 9999  
1 1000 8363 9999
```

=== Code Execution Successful ===

3. Online Store Sales Ranking

```
#include <stdio.h>
void swap(int* a, int* b)
{
    int t = *a;
    *a = *b;
    *b = t;
}

int partitionDescending(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = (low - 1);
    for (int j = low; j <= high - 1; j++)
    {
        if (arr[j] > pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

void quickSortDescending(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partitionDescending(arr, low, high);
        quickSortDescending(arr, low, pi - 1);
        quickSortDescending(arr, pi + 1, high);
    }
}

int main()
```

```

{
    int n;
    scanf("%d", &n);
    int sales[n];
    for (int i = 0; i < n; i++) scanf("%d", &sales[i]);
    quickSortDescending(sales, 0, n - 1);
    for (int i = 0; i < n; i++) printf("%d ", sales[i]);
    printf("\n");
    printf("Top 3: %d %d %d\n", sales[0], sales[1], sales[2]);
    int average = (sales[0] + sales[1] + sales[2]) / 3;
    printf("Average: %d\n", average);
    return 0;
}

```

Output

```

6
101 737 999 204 599 299
999 737 599 299 204 101
Top 3: 999 737 599
Average: 778

```

=== Code Execution Successful ===

****THE END****